Custom Reinforcement Learning Agent Using Gymnasium

Topics in AI — Assignment No. I (CLO-I) | FrozenLake-v1 with Tabular Q-Learning

Student Name: Rimsha Malik **Registration No:** FA23-BAI-034

1. Problem Description

This project implements a reinforcement learning (RL) agent for the FrozenLake-v1 environment provided by Farama Gymnasium. The agent starts on a grid and must reach the goal while avoiding holes. The environment is slippery, so an action may not always move the agent in the intended direction. The observation space contains 16 discrete states and the action space contains 4 actions. The reward used in this experiment is 1 for reaching the goal and 0 otherwise.

2. Methodology

The agent uses tabular Q-Learning, a model-free, off-policy temporal-difference algorithm. A Q-table stores an estimated value for each state-action pair. At the start of training, Q-values are initialized to zero. Actions are selected using an epsilon-greedy strategy: with probability epsilon, a random action is selected; otherwise, the action with the largest Q-value is selected. Epsilon is gradually reduced to shift from exploration toward exploitation.

The update rule is: $Q(s,a) \leftarrow Q(s,a) + \alpha\,[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$. Here, α is the learning rate, γ is the discount factor, r is the immediate reward, s is the current state, a is the chosen action, and s' is the next state. For terminal transitions, the future-value term is set to zero.

3. Implementation Architecture

- **agent.py:** defines the QLearningAgent class, Q-table, action selection, Q-value updates, epsilon decay, and save/load methods.
- **main.py:** creates FrozenLake-v1, trains the agent, plots moving-average rewards, saves the Q-table, and evaluates the greedy policy.
- **utils.py:** loads the saved Q-table and records an RGB-frame GIF of one policy rollout.
- **FrozenLake_RL_Assignment.ipynb:** displays environment/library information, the saved Q-table shape, training graph, and evaluation summary.

4. Experimental Setup and Hyperparameters

Parameter	Value
Environment	FrozenLake-v1 (is_slippery=True)
Training episodes	50,000
Learning rate (α)	0.1
Discount factor (γ)	0.99
Initial epsilon	1.0
Epsilon decay	0.9999 per episode
Minimum epsilon	0.01
Evaluation episodes	100
Q-table dimensions	16 × 4

5. Results and Discussion

During training, the success rate reported over successive blocks of 1,000 episodes generally increased, reaching 67.4% in the final 1,000-episode block. The saved moving-average reward graph also shows an overall upward trend, with fluctuations caused by the stochastic slippery dynamics and exploration. In a separate evaluation of 100 episodes using the greedy policy, 71 episodes reached the goal, giving a success rate of 71.0%. The saved Q-table has shape (16, 4). A recorded rollout also reached the goal (Goal reached: True).

The evaluation result is encouraging but does not indicate perfect performance. Because the environment is stochastic, the same policy may succeed in some episodes and fail in others. Performance can vary with the random seed and training configuration. Further experiments could compare learning rates, epsilon-decay schedules, and training durations using repeated evaluation runs.

6. Conclusion

A tabular Q-Learning agent was implemented and trained in Farama Gymnasium's FrozenLake-v1 environment. The project includes Python source files, a saved Q-table, a training-reward visualization, a GIF rollout, and a Jupyter Notebook. The

measured evaluation success rate was 71% over 100 episodes. This experiment demonstrates how Q-Learning combines reward-based updates with exploration and exploitation to learn a policy for a discrete decision-making task.

Reproducibility note: Results in this report reflect the console output provided for the final 50,000-episode run. The training graph and model file should be submitted alongside this report.